I built an automated content pipeline that generates LinkedIn-style posts from structured brand inputs and routes them through a controlled approval flow before publishing.

The architecture has three layers.

First, a FastAPI microservice exposes a linkedin endpoint and a health endpoint. It accepts topic, audience, and tone, then returns a formatted post payload.

Second, an n8n orchestration workflow handles end-to-end logic: Brand Config, AutoGen Microservice call, Compose Final, Approval Gate, and delivery routing.

Third, Slack is used as a dry-run destination so we can validate output safely before enabling live LinkedIn publishing.

The key technical decisions were about reliability and safety.

I normalized all generated output into one field called final_post so downstream nodes are deterministic.

I added an approval gate to block weak or empty content and route failed cases to a rejection path instead of publishing.

I also added a mode-based delivery strategy so the same workflow supports dry-run and live mode without rewiring.

For validation, I tested each component independently first, then tested node-by-node, then ran repeated end-to-end executions with different topics.

Success criteria were consistent output shape, successful delivery events, and no broken expression mappings.

Operationally, I also handled a VM outage during development. After recovery, I revalidated container health, service health, and workflow execution, which confirms the system is not only functional but recoverable.

So the final result is an automation pipeline that is modular, testable, and ready for controlled go-live.

Strong Q and A answers

Why separate microservice and workflow instead of doing everything in n8n?

Separation keeps business logic in code and orchestration in workflow. That improves maintainability, testing, and future portability.

Why is Slack dry-run important?

It reduces production risk. We can inspect content quality and routing behavior before posting publicly.

How do you prevent bad content from posting?

Approval Gate checks final_post validity and sends failures to a non-publish path.

What was the hardest bug you fixed?

Expression mapping mode mismatches. Some values were being passed as literal templates. I fixed that by using correct expression handling and normalized outputs.

How did you validate reliability?

Three-stage validation: service-level checks, node-level execution, and repeated full-run tests with varied inputs.

What happens if infrastructure fails?

The recovery process is documented: restart VM, verify n8n container, verify microservice health, run one end-to-end smoke test.